# CS 3110 Programming Guidelines

The OCaml community maintains a set of **OCaml Programming Guidelines (https://ocaml.org/docs/guidelines)** that represents a broad consensus about readable, idiomatic OCaml. This page highlights the practices most relevant to CS 3110 and adds a few course-specific expectations. Each assignment handout will identify the section or sections that apply.

These guidelines are here to help you write code that is easy for you and other people to understand. We are looking for reasonable, consistent choices—not one perfect way to organize every program.

## A0 and A1

### Build Without Errors or Warnings

Run the following command from the top-level directory of your project regularly while you work and once more immediately before you submit:

```
dune build
```

This command asks Dune to compile and check the entire project. It should finish without reporting any errors **or warnings**. An error means that Dune could not successfully build the project. A warning means that the compiler found something suspicious, such as unused or incomplete code. A program that builds but still produces warnings is not finished: resolve every warning before you submit, rather than ignoring or suppressing it.

### Formatting

The course installation instructions configured VS Code to run `ocamlformat` automatically whenever you save an OCaml file. If that setup is working, saving `bin/main.ml` should keep it formatted. You can also run the formatter manually:

1. Open `bin/main.ml`.
2. Open the Command Palette with **Command+Shift+P** on macOS or **Control+Shift+P** on Windows and Linux.
3. Choose **Format Document**.
4. Save the file.

If the formatter chooses a different layout than you expected, take a moment to make sure OCaml is parsing the expression the way you intended.

## Naming and Decomposition

Use `snake_case` for function, parameter, and local-value names. Choose names that help a reader understand the role of each value. Short names are fine when their meaning is obvious in a very small scope, but avoid names such as `l` and `o`, which are easily confused with `1` and `0`.

Break a long function into helpers when doing so gives a meaningful name to part of the computation or makes the control flow easier to follow. Likewise, avoid copying and pasting a substantial piece of code when a small helper function could express the shared idea. Not every repeated operator or one-line expression needs an abstraction.

## Write Function Specifications

Every top-level function you define in an `.ml` file must have an OCamldoc specification. For A0 and A1, put it immediately above the function's definition. The A2 section below gives another placement option. An OCamldoc comment begins with two asterisks: `(** ... *)`.

Write each specification when you write the function, rather than adding all the comments at the end. OCaml treats `(** ... *)` as documentation for a nearby program element, so an OCamldoc comment in the wrong place can produce a compiler warning and may cause `dune build` to fail. Use an ordinary `(* ... *)` comment for a free-standing note that does not document the definition immediately below it. After adding or moving comments, run `dune build` again.

A specification is documentation, not a type annotation. You do not need to repeat a function's inferred type in its specification unless doing so would make the behavior clearer.

A specification describes what an application of the function means. It should mention:

- what the parameters represent;
- what value the function returns, or what effect it performs; and
- any precondition that callers must satisfy, if there is one.

Describe the function's behavior rather than narrating its implementation. For example:

```
(** [clamp n] is [n] limited to the inclusive range 0 to 100. *)
let clamp n = ...
```

For a function used primarily for output, describe the effect:

```
(** [print_status name fullness happiness energy] prints the current status
    of the pet named [name]. *)
```

```
let print_status name fullness happiness energy = ...
```

Local helper functions and ordinary local-value definitions do not require OCamldoc specifications. You may still use an ordinary comment, written `(* ... *)`, when it would help a reader understand a non-obvious decision.

## Cite Outside Sources

If you adapt code or an algorithm from a source outside the current course materials, add a brief source comment with enough information for a reader to find the original. The exact citation format is not important. You do not need to cite the current semester's textbook, lectures, worksheets, or assignment materials. AI use and collaboration are governed by the policies in the assignment handout.

# A2

## Function Specification Placement

Continue to write a function specification for each top-level function in a `.ml` file. Also write a specification for each function in the corresponding `.mli` file.

Many OCaml libraries instead put public specifications in `.mli` files, and omit the corresponding specification in the `.ml` files.

For CS 3110, we now prefer to write specifications in **both** the `.ml` and `.mli` files. *This is a change from previous semesters, so course staff might be confused on this point. We are making a change based on experience that we have gained.*

The advantage to this approach is that specifications are readily available in both the implementation and the interface files, thus making it easier for you to find and reference them, and ensuring that you always get an editor hover-over with the specification. A secondary benefit is that AI tools can more easily understand the code when the specifications are available in both locations.

The drawback is that maintaining the specifications in both files requires more effort and can lead to inconsistencies if the specifications are not kept in sync. If that truly bothers you, you have our blessing to stick with the traditional approach: put public specifications in `.mli` files, omit just the corresponding specifications in the `.ml` files, and write specifications for all the *other* functions in the `.ml` files.

## Test to Completion

Ensure that `dune test` completes all your test cases. That doesn't necessarily mean that every test case passes, but it does mean that every test case gets a chance to run. We do not want to see tests being skipped or ignored due to compilation errors, uncaught exceptions, or other issues.

## Be Careful with File Paths

For your program to run correctly on another computer, including the smoketester, use *relative* file paths rather than paths tied to your own computer. A path starting with `/`, such as `/home/<your netid>/3110/a1`, is absolute and may not exist on another computer. A path starting with `~`, such as `~/3110/a1`, is not an absolute path: a shell may expand `~` to your home directory, but OCaml file operations do not do so automatically. Neither kind of path is suitable for a file your submitted program must read.

Also, be careful about capitalization of file names – for example, `file.txt` vs. `file.TXT`. Those are technically different file names. It's possible that your computer's file system might ignore capitalization, but many other computers will not.